

MERGE SORT

Divide & Conquer Sorting Algorithm

What is Merge Sort?

- ✓ A divide-and-conquer sorting algorithm
- ✓ Divides array into smaller sub-arrays
- ✓ Recursively sorts each sub-array
- ✓ Merges sorted sub-arrays back together

How It Works - Two Main Phases

DIVIDE PHASE

- 1. Split array into halves
- 2. Recursively divide halves
- 3. Continue until single elements
- 4. Each single element is sorted

MERGE PHASE

- 1. Compare sorted elements
- 2. Place smaller element first
- 3. Merge sub-arrays gradually
- 4. Result: fully sorted array

Visual Example: [38, 27, 43, 10]

Input:

[38, 27, 43, 10]

DIVIDE:

[38, 27, 43, 10]

[38, 27] [43, 10]

[38] [27] [43] [10]

MERGE:

[27, 38] [10, 43]

[10, 27, 38, 43]

OUTPUT: [10, 27, 38, 43] ✓

Code Structure - Two Functions

merge() Function

- Merges two sorted sub-arrays:
- • Creates temp arrays L[] & R[]
- • Copies left & right portions
- • Compares elements pairwise
- • Places smaller element first
- • Copies remaining elements

mergeSort() Function

- Recursively divides array:
- • Base case: $l < r$ (not single)
- • Find middle index m
- • Recursively sort left half
- • Recursively sort right half
- • Merge the two sorted halves

Code Breakdown: merge() Function

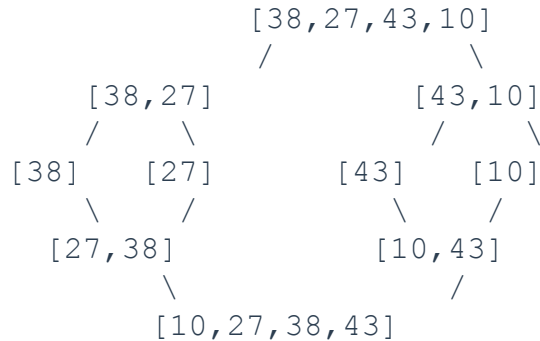
```
void merge(int arr[], int l, int m, int r) {  
  
    int n1 = m - l + 1;           // Left array size  
    int n2 = r - m;               // Right array size  
    int L[n1], R[n2];            // Temp arrays  
  
    for (i = 0; i < n1; i++)      // Copy left portion  
        L[i] = arr[l + i];  
    for (j = 0; j < n2; j++)      // Copy right portion  
        R[j] = arr[m + 1 + j];  
  
    i = 0, j = 0, k = l;  
    while (i < n1 && j < n2) {    // Compare & merge  
        if (L[i] <= R[j]) {  
            arr[k] = L[i]; i++;  
        } else {  
            arr[k] = R[j]; j++;  
        }  
        k++;  
    }  
}
```

Code Breakdown: mergeSort() Function

```
void mergeSort(int arr[], int l, int r) {  
  
    if (l < r) {                // Base case check  
        int m = l + (r - l) / 2; // Find middle  
        mergeSort(arr, l, m);    // Sort left half  
        mergeSort(arr, m + 1, r); // Sort right half  
        merge(arr, l, m, r);    // Merge halves  
    }  
}
```

Key: $(r - l) / 2$ avoids overflow | Base case: Single element is sorted | Recursive until all divided

Recursion Tree: [38, 27, 43, 10]



$\log_2(n) = 2$ levels | n work per level = $O(n \log n)$

Merge Process: Detailed Steps

Merging [27, 38] and [10, 43]:

1

Compare: 27 vs 10

Take 10 → [10, _, _, _]

2

Compare: 27 vs 43

Take 27 → [10, 27, _, _]

3

Compare: 38 vs 43

Take 38 → [10, 27, 38, _]

4

Compare: —

Take 43 → [10, 27, 38, 43]

Complexity Analysis

Time Complexity

Best: $O(n \log n)$

Average: $O(n \log n)$

Worst: $O(n \log n)$

Dividing: $\log n$ levels

Per level: n comparisons

Space Complexity

Auxiliary Space: $O(n)$

Temp arrays: $O(n)$

Recursion stack: $O(\log n)$

Total: $O(n)$

Why $O(n \log n)$?

- Divide phase: $\log n$ levels (halving each time)
- Work per level: n comparisons and copies
- Total work: $n \times \log n = O(n \log n)$

Complete Program: main() Function

```
int main() {  
    int arr[] = {38, 27, 43, 10};  
    int arr_size = sizeof(arr) / sizeof(arr[0]); // = 4  
  
    mergeSort(arr, 0, arr_size - 1);  
  
    for (int i = 0; i < arr_size; i++)  
        printf("%d ", arr[i]);  
    printf("\n");  
  
    return 0;  
}
```

Output: 10 27 38 43

Key Takeaways

- ✓ Merge Sort = Divide-and-Conquer approach
- ✓ Two phases: Divide (recursive) → Merge (combine)
- ✓ Time: $O(n \log n)$ - guaranteed in all cases
- ✓ Space: $O(n)$ - requires temporary arrays
- ✓ Stable: Maintains relative order of equal elements
- ✓ Best for: Large datasets, external sorting, linked lists
- ✓ Downside: Extra memory, slower on small data